MCCB-gpu: A Massively Parallel GPU-Accelerated Checkerboard Monte Carlo Engine for Colloidal and Molecular Systems with Advanced Sampling and In-Memory Hybrid Molecular Dynamics

Antonio Díaz-Pozuelo[a], Enrique Lomba[a,b,*], Eva González Noya[a]

[a]*Instituto de Química Física Blas Cabrera (IQF-CSIC), CSIC, Serrano 119, 28006 Madrid, Spain*
[b]*Grupo NAFOMAT, Facultade de Física, Universidade de Santiago de Compostela, Santiago de Compostela, Spain*

Abstract

We present `MCCB-gpu`, a high-performance, massively parallel Monte Carlo (MC) simulation framework written in CUDA Fortran and modern Fortran 2008 for the simulation of soft-matter, colloidal, and small molecular systems on graphics processing units (GPUs). Utilizing a three-dimensional checkerboard spatial domain decomposition into eight non-interfering sublattices, `MCCB-gpu` executes conflict-free, concurrent trial moves across thousands of CUDA threads while strictly preserving detailed balance. The code supports canonical (NVT) and isobaric-isothermal (NpT) ensembles for a wide spectrum of interaction potentials, including hard spheres (HS), Lennard-Jones (LJ), arbitrary user-tabulated potentials, angular patchy particles (Lennard-Jones-Gauss), and site-site patchy models for colloids and biomolecular condensates [1, 2]. To bypass kinetic jamming and glassy arrest in dense systems, or entropic bottlenecks in dilute associating systems, `MCCB-gpu` integrates three advanced sampling algorithms: (1) an Association Volume Bias Monte Carlo (AVBMC) engine that dynamically targets cluster boundaries identified via local geometric surface dipole asymmetry ($\eta_i \geq 0.5$), coupled to a parallel GPU implementation of G-DBSCAN (taken from the companion `trj_analysis` package), making the code suitable to study complex cluster formation problems or nucleation; (2) GPU checkerboard particle identity swaps ($A \leftrightarrow B$) for multicomponent mixtures, targeting the study of glass formation; and (3) an in-memory persistent-state Hybrid Monte Carlo (HMC) interface coupling `MCCB-gpu` with GPU-accelerated LAMMPS molecular dynamics. We present analytical proofs and machine-precision numerical verifications of microscopic reversibility ($\delta_{\mathrm{DB}} < 10^{-15}$), thermodynamic consistency tests across NpT and NVT ensembles, and extensive scaling benchmarks. Comparing against an optimized serial Monte Carlo CPU code (`gpMC`) and accounting for trial move throughput normalization, `MCCB-gpu` delivers speedups of up to **61.4**× in NVT and **68.8**× in NpT on an NVIDIA RTX PRO 4500 GPU for samples $\approx 10^5$ particles, scaling seamlessly up to millions of particles with a performance comparable to LAMMPS GPU-accelerated Molecular Dynamics.

Keywords: Monte Carlo simulation, GPU acceleration, Checkerboard domain decomposition, Association Volume Bias Monte Carlo (AVBMC), G-DBSCAN, Identity swap moves, Hybrid Monte Carlo, Patchy colloids

PROGRAM SUMMARY

Program Title: `MCCB-gpu`

CPC Library link to program files: (to be assigned by CPC)

Developer's repository link: `https://github.com/adpozuelo/MC_Checkerboard`

Licensing provisions: GNU General Public License v3.0 (GPLv3)

Programming language: Fortran 2008, CUDA Fortran, ISO_C_BINDING, Python 3

Computer: Modern x86_64 HPC clusters, workstations with NVIDIA GPUs (Compute Capability ≥ 6.0, tested up to Blackwell, Ada Lovelace, Ampere, Volta, Pascal)

Operating system: Linux (x86_64; RHEL, CentOS, Ubuntu, Debian)

RAM: 1–4 GB host RAM, 2–32 GB device VRAM (scales with particle count)

Classification: 7.7 Molecular Dynamics and Particle Methods, 4.8 Statistical Mechanics.

External libraries: NVIDIA HPC SDK (NVHPC ≥ 22.*x* with CUDA ≥ 11.8), NetCDF-Fortran (required for compressed trajectories), LAMMPS shared C-library (`liblammps`, optional, for Hybrid Monte Carlo).

Nature of problem: Equilibrium Monte Carlo sampling of dense, large-scale, or strongly associating colloidal fluids, patchy nanoparticles, and multicomponent molecular mixtures across canonical (NVT) and isobaric-isothermal (NpT) ensembles. Conventional single-particle MC algorithms suffer from severe kinetic arrest due to steric caging, entropic barriers in cluster nucleation, and compositional jamming.

Solution method: Massive parallelism via 3D checkerboard domain decomposition partitioning the box into 8 independent sublattices evaluated concurrently by CUDA thread blocks. To overcome physical kinetic arrest, the code incorporates: (1) Association Volume Bias Monte Carlo (AVBMC) guided by local geometric surface dipole asymmetry $\eta_i \geq \eta_{\mathrm{thresh}}$ coupled with GPU G-DBSCAN cluster identification (from the companion

*Corresponding author.
Email address: `enrique.lomba@csic.es` (Enrique Lomba)

`trj_analysis` library); (2) Intra-cell and long-range cross-cell particle identity swaps ($A \leftrightarrow B$) for multicomponent mixtures ($N_{\text{species}} \geq 2$); and (3) In-memory persistent-state Hybrid Monte Carlo (HMC) coupling with GPU-accelerated LAMMPS.

Restrictions: Currently only Lennard-Jones units are implemented. Molecular structures are described by interaction sites, and orientations by quaternions. As it stands the code runs efficiently only for small molecules.

Running time: Milliseconds per sweep; e.g., $\sim$ 4.6 ms/sweep for N = 125,000 particles on NVIDIA RTX PRO 4500 (delivering up to 68.8× algorithmic throughput speedup over optimized serial CPU code). Larger throughput to be expected for larger samples.

Additional comments: Examples and validation routines are distributed with the software. Atomic input/restart configurations and trajectories are designed to be compatible with LAMMPS Molecular Dynamics package [3] and with the companion trajectory analysis tool `trj_analysis` [4].

1. Introduction

Monte Carlo (MC) computer simulations represent a cornerstone of computational physics, chemistry, and materials science [5, 6]. By stochastically sampling configurations according to the Boltzmann distribution, MC methods enable the calculation of equilibrium thermodynamic observables, phase diagrams, free energy landscapes, and structural correlation functions without requiring numerical integration of equations of motion, bypassing kinetic bottlenecks that plague Molecular Dynamics approaches in many instances. In the realm of soft matter, complex fluids, and colloidal nanotechnology, MC simulations are indispensable for investigating self-assembly, crystallization, gelation, and liquid-liquid phase separation in isotropic fluids [7], patchy colloids [8, 9, 10, 1], and dense multicomponent glass-forming liquids [11, 12].

Despite their theoretical generality, standard MC algorithms face two narrow computational bottlenecks:

1. **Parallelization Constraints**: Unlike Molecular Dynamics (MD), where forces on all N particles can be computed simultaneously in parallel [13, 3], standard Metropolis MC algorithms evaluate sequential single-particle trial displacements, i.e., they are serial by construction. Modifying the position or orientation of particle i alters the local potential energy field, thereby invalidating concurrent moves of any neighboring particle j within the interaction cutoff distance r_{cut}. This sequential dependency has hindered so far the efficient exploitation of massively parallel graphics processing units (GPUs), in contrast with their widespread usage in Molecular Dynamics codes.
2. **Physical Kinetic Arrest**: In condensed systems, standard random particle translations and rotations resulting from either a Monte Carlo sweep or a Molecular Dynamics integration step frequently become trapped in local free-energy minima:
    - *Steric Caging and Jamming*: In dense liquids and glass formers, particles are enclosed by rigid cages formed by nearest neighbors; translational displacements that yield reasonable acceptance rates (30–50%) are constrained to fractions of an angstrom, leading to extremely slow diffusive relaxation [11], and the same applies to MD steps.
    - *Entropic Bottlenecks in Dilute Associating Clusters*: In dilute patchy colloidal fluids ($V_{\text{box}} \gg V_{\text{clusters}}$), binding energies are strong ($\sim 10\,k_BT$). Gas-phase monomers spend huge numbers of Monte Carlo sweeps diffusing through empty space before randomly encountering an active cluster binding patch, while bonded particles rarely evaporate, which practically stops cluster size equilibration [14, 15].
    - *Compositional Freezing*: In multicomponent mixtures with non-additive diameters or asymmetric attractions, chemical species separation is arrested unless particles can interchange chemical identities without spatial diffusion [11, 12].

To address parallelization, spatial domain decomposition techniques—most notably the checkerboard algorithm—have emerged as an effective paradigm for GPU acceleration [16, 17]. By partitioning the simulation box into cells with linear dimensions larger than the potential cutoff r_{cut} and assigning cells to interleaved, mutually non-interacting sublattices, thousands of trial moves can be proposed and evaluated concurrently by independent CUDA thread blocks without violating detailed balance.

In this paper, we introduce `MCCB-gpu` (Monte Carlo Checker-Board on GPU), an open-source, high-throughput simulation package written in CUDA Fortran and modern Fortran 2008. `MCCB-gpu` combines a 3D checkerboard GPU cellular engine with:

- Full canonical (NVT) and isobaric-isothermal (NpT) ensemble sampling, with isotropic and anisotropic box volume trial moves.
- Broad potential support: athermal Hard Spheres (HS), continuous Lennard-Jones (LJ), arbitrary user-tabulated potentials (TABLE), angular patchy particles (Lennard-Jones-Gauss, LJG), and site-site patchy models for colloidal fluids (Palaia SSP model [1]) and biomolecular condensates (Sanchez-Burgos CSW/PHS model [2]) with quaternion rotational kinematics.
- Association Volume Bias Monte Carlo (AVBMC) [18, 19] guided by local geometric surface dipole asymmetry ($\eta_i \geq 0.5$) and coupled to a GPU implementation of G-DBSCAN (derived from the companion `trj_analysis` package [20, 4]).
- GPU-accelerated intra-cell and long-range cross-cell particle identity swaps ($A \leftrightarrow B$) for arbitrary multicomponent mixtures ($N_{\text{species}} \geq 2$).
- A high-efficiency, persistent in-memory Hybrid Monte Carlo (HMC) coupling layer with the LAMMPS molecular dynamics engine [3], accelerating coordinate transfer by 287× over disk I/O.

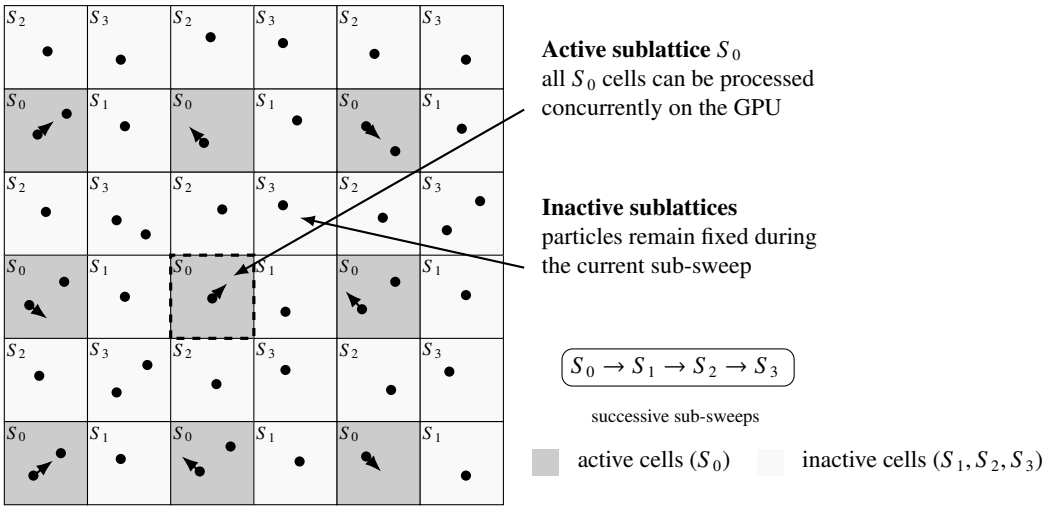


Figure 1: Schematic representation of the four-sublattice checkerboard Monte Carlo algorithm in two dimensions.

Several GPU-accelerated Monte Carlo simulation packages have been developed in recent years. The GPU Optimized Monte Carlo (GOMC) package [21, 22] provides a general framework for molecular simulations and supports canonical (NVT), isothermal–isobaric (NpT), grand-canonical (μVT), and Gibbs ensembles, together with a broad range of all-atom, united-atom, and coarse-grained force fields and advanced sampling schemes such as configurational-bias and multi-particle Monte Carlo moves. In contrast, `MCCB-gpu` is primarily targeted at simple and multicomponent fluids, including additive and non-additive hard-sphere mixtures, Lennard–Jones systems, arbitrary tabulated isotropic potentials, and anisotropic patchy particles. As mentioned, a distinctive feature of `MCCB-gpu` is that trial moves themselves are executed concurrently on the GPU through a spatial checkerboard decomposition, rather than limiting GPU acceleration to the evaluation of interactions associated with sequentially proposed moves. This strategy follows that of the massively parallel Metropolis Monte Carlo approach introduced for hard particles and subsequently implemented in the HPMC component of HOOMD-blue [23]. `MCCB-gpu` extends this general approach to energetic isotropic and orientation-dependent interactions and provides GPU-parallel particle translations, rotations, and identity-swap moves, as well as aggregation-volume-bias Monte Carlo sampling for strongly associating systems. Thus, while GOMC offers considerably broader ensemble coverage, molecular force-field support, and a more mature simulation ecosystem, `MCCB-gpu` occupies a complementary niche focused on massively concurrent off-lattice Monte Carlo simulations of simple mixtures and anisotropic patchy-particle systems. Importantly, `MCCB-gpu` builds upon and extends the original CUDA-C GPU Monte Carlo simulation code developed by E. G. Noya [24], which was originally introduced and applied to the self-assembly and discovery of colloidal quasicrystals [25, 26, 27].

The remainder of this manuscript is organized as follows: Section 2 details the mathematical formulations and proofs of detailed balance for the checkerboard decomposition, AVBMC, surface geometry identification, identity swaps, and HMC. Section 3 outlines the program architecture, input namelists, and file formats. Section 4 presents thermodynamic consistency tests. Section 5 discusses scaling and computational performance. Section 6 concludes the paper.

2. Theoretical Formulation and Algorithms

2.1. 3D Checkerboard Domain Decomposition

Consider a simulation domain of volume $V = L_x \times L_y \times L_z$ containing N interacting bodies subject to periodic boundary conditions (PBC). The interaction potential U_{ij} vanishes beyond a finite spherical cutoff distance r_{cut}, such that particles separated by $r_{ij} \geq r_{\text{cut}}$ exert zero force and have zero pairwise energy.

To enable conflict-free concurrent evaluation on the GPU, `MCCB-gpu` discretizes the simulation volume into a regular Cartesian grid of $M_x \times M_y \times M_z = N_{\text{cells}}$ cells. The linear cell dimensions $L_{c,\alpha}$ satisfy:

$$L_{c,\alpha} = \frac{L_\alpha}{M_\alpha} \geq r_{\text{cut}}, \quad \alpha \in \{x, y, z\} \tag{1}$$

Cells are indexed by their discrete spatial coordinates (c_x, c_y, c_z) with $c_\alpha \in \{0, \ldots, M_\alpha - 1\}$. Each cell is assigned a color based on its 3D parity:

$$C(c_x, c_y, c_z) = (c_x \bmod 2) + 2(c_y \bmod 2) + 4(c_z \bmod 2) \tag{2}$$

with $C \in \{0, \ldots, 7\}$. This partitions the N_{cells} cells into 8 interleaved sublattices $\mathcal{S}_0, \ldots, \mathcal{S}_7$. Crucially, for any two distinct cells $C_1, C_2 \in \mathcal{S}_\alpha$ belonging to the same sublattice, their center-to-center separation along any Cartesian axis is at least $2L_{c,\alpha} \geq 2r_{\text{cut}}$. Therefore, no particle in cell C_1 can interact with any particle in cell C_2. A schematic representation of the algorithm in 2D is shown in Figure 1. We see that the simulation domain is partitioned into spatial cells, each assigned to one of four sublattices S_0, S_1, S_2, or S_3 according to the parity of its cell coordinates. Specifically, in 2D the sublattice index is

Table 1: **Control Parameters Namelist (`&Control_Params`)**

Variable	Type	Default	Description / Physical Function
`istep_ini`	Integer	1	Initial simulation sweep index (automatically updated in restarts).
`istep_fin`	Integer	1000	Final simulation sweep index.
`Neq`	Integer	0	Number of equilibration sweeps (virial statistics excluded during equilibration).
`Nmove`	Integer	125	Trial moves attempted per active cell during each checkerboard sub-sweep.
`Nsave`	Integer	100	Frequency (in sweeps) for writing thermodynamic averages to `run-data.dat`.
`Ndump`	Integer	1000	Frequency (in sweeps) for appending frames to the trajectory file.
`Nrestart`	Integer	5000	Frequency (in sweeps) for writing restart checkpoints (`data.restart`).
`traj_format`	Character	`'netcdf'`	Trajectory format: `'netcdf'` (binary NetCDF), `'plain'` (text XYZ/LAMMPS), or `'both'`.
`data_file`	Character	`'data.atoms'`	Path to the initial atomic configuration file (LAMMPS data format).
`Npart_types`	Integer	1	Number of distinct chemical species in the mixture (N_{species}).
`ncluster`	Integer	0	Frequency (in sweeps) for DBSCAN cluster analysis (0 = disabled).
`rcl`	Real	1.8	Radial neighbor connectivity threshold (r_{cl}) for cluster formation.
`minPts`	Integer	3	Minimum neighbor count for DBSCAN core node classification.
`asym_threshold`	Real	0.5	Dipole asymmetry threshold (η_{thresh}) for cluster surface identification.
`cluster_types`	Integer(10)	-1	Species types included in cluster search (-1 = all species).
`avbmc`	Logical	`.false.`	Master flag to enable Association Volume Bias Monte Carlo moves.
`n_avbmc`	Integer	10	Frequency (in sweeps) of lightweight AVBMC exchange moves in Tier 1.
`ncluster_init`	Integer	500	Periodicity (in sweeps) of full DBSCAN nucleation checks in Tier 1.
`cluster_threshold`	Integer	5	Size threshold (S_{thresh}) to transition from Tier 1 to Tier 2.
`bound_min_neighbors`	Integer	1	Minimum coordination (z_{bound}) for bound particle classification in Tier 1.
`border_criterion`	Character	`'asymmetry'`	Border identification method in Tier 2: `'asymmetry'` or `'energy'`.
`avbmc_k_trials`	Integer	10	Number of bulk Rosenbluth trial positions (K) per AVBMC attempt.
`avbmc_max_trials`	Integer	50	Maximum number of pair attempts per AVBMC invocation.
`lammps`	Logical	`.false.`	Master flag to enable in-memory Hybrid Monte Carlo moves via LAMMPS.

Table 2: **Monte Carlo Parameters Namelist (`&MC_Params`)**

Variable	Type	Default	Description / Physical Function
`temp0`, `temp1`	Real	1.0, 1.0	Initial and final temperatures (T^* or Kelvin). Linear ramp if `temp0` ≠ `temp1`.
`npt`	Logical	`.false.`	Master flag for isobaric-isothermal (NpT) ensemble sampling.
`pres`	Real	0.0	Target pressure (P^* in reduced units or atmospheres in real units).
`Nvolf`	Integer	1	Frequency (in MC sweeps) of trial volume moves when `npt = .true.`
`hmax`	Real	0.1	Maximum trial displacement distance along Cartesian axes.
`omax`	Real	0.1	Maximum trial quaternion angular rotation parameter.
`vmax`	Real(3)	0.01	Maximum trial volume scaling factor along Cartesian axes.
`displ_update`	Logical	`.false.`	Dynamically adjust `hmax` and `omax` to achieve 40% acceptance.
`seed`	Integer	123456	Pseudo-random number generator seed.
`swap_moves`	Logical	`.false.`	Enable GPU checkerboard particle identity swaps ($N_{\text{part_types}} \geq 2$).
`Nswap`	Integer	1	Number of swap sub-passes executed per swap invocation.
`Nswapf`	Integer	1	Frequency (in MC sweeps) of identity swap moves.

defined as $C(c_x, c_y) = (c_x \bmod 2) + 2(c_y \bmod 2)$, with a cell (c_x, c_y) belonging to S_α with $\alpha = C(c_x, c_y)$. During a given sub-sweep only one sublattice is active; here, S_0 is shown in darker shading. Particles located in different active cells can perform trial moves simultaneously because active cells are spatially separated, while particles belonging to the complementary sublattices remain fixed. Successive sub-sweeps activate S_1, S_2, and S_3. The two-dimensional construction is the direct analogue of the $2 \times 2 \times 2$ eight-sublattice decomposition introduced above.

A single Monte Carlo sweep consists of 8 sequential sub-sweeps (one per sublattice $\mathcal{S}_k$). During sub-sweep k, a CUDA kernel is launched where each thread block is assigned to an active cell $C \in \mathcal{S}_k$. The thread block executes N_{move} trial moves within cell C. To evaluate the change in potential energy ΔE, the threads examine particles residing in cell C and its 26 nearest-neighbor cells in the 3D grid. Because neighbor cells belong to different sublattices $\mathcal{S}_{k'} \neq \mathcal{S}_k$, their particle coordinates and occupancies are held strictly constant during sub-sweep k. This guarantees zero read-write race conditions across CUDA blocks, ensuring strict detailed balance without spinlocks or thread synchronization.

2.2. Canonical (NVT) and Isobaric-Isothermal (NpT) Sampling

In the canonical (NVT) ensemble, trial moves consist of random particle displacements $\mathbf{r}_i \to \mathbf{r}_i + \delta\mathbf{r}$ with $\delta r_\alpha \in [-h_{\max}, h_{\max}]$.

Table 3: **Potential Parameters Namelist (`&Potential_Params`)**

Variable	Type	Default	Description / Physical Function
`model`	Character	`'LJ'`	Interaction potential: `'HS'`, `'LJ'`, `'TABLE'`, `'LJG'` (angular), `'SSP'` (Palaia site-site), or `'SB'` (Sanchez-Burgos site-site).
`table_mc`	Logical	`.false.`	Use Catmull-Rom interpolated potential tables on the GPU.
`table_file_mc`	Character	`''`	Base prefix for MC potential table files (e.g., `'pot'` for `pot11.dat`).
`Bool_tor`	Logical	`.false.`	Enable torsional angular patch modulation in LJG models.
`sigp_factor`	Real	0.1	Patch width scaling factor in Palaia SSP models.
`Rcp_factor`	Real	0.3	Patch radial cutoff scaling factor in Palaia SSP models.
`Rc_factor`	Real	2.0	Core potential cutoff scaling factor in Palaia SSP models.
`patch_radial_factor`	Real	0.5	Distance from particle center to patch site (r_p/σ).
`eps_R`	Real	0.6667	PHS core repulsive energy scale parameter (ϵ_R) in SB model.
`eps_CSW`	Real	11.1111	CSW patch attractive well depth ($\epsilon_{\text{CSW}} = 1/T^*$) in SB model.
`rw_CSW`	Real	0.12	CSW patch attractive well width (r_w/σ) in SB model.
`alpha_CSW`	Real	0.005	CSW patch well smoothing parameter (α/σ) in SB model.
`rc_CSW`	Real	0.20	CSW patch interaction radial cutoff (r_c/σ) in SB model.

Table 4: **LAMMPS Hybrid Monte Carlo Parameters Namelist (`&Lammps_Params`)**

Variable	Type	Default	Description / Physical Function
`timestep`	Real	0.0025	MD integration timestep for HMC trajectory segments (Δt).
`Nmd`	Integer	50	Number of microcanonical MD steps executed per HMC trajectory segment.
`hmc_freq`	Integer	10	Cadence (in MC sweeps) between successive HMC trajectory proposals.
`neigh_skin`	Real	0.3	Neighbor list skin distance in LAMMPS distance units.
`thermo_freq`	Integer	0	Frequency of internal LAMMPS thermodynamic logging (0 = disabled).
`use_gpu`	Logical	`.true.`	Accelerate LAMMPS pair evaluation via the GPU package (`-pk gpu 1`).
`gpu_id`	Integer	-1	GPU device ID for LAMMPS (-1 = inherit `MCCB-gpu` active device).
`table_lammps`	Logical	`.true.`	Force tabulated force representation in LAMMPS (`pair_style table`); mandatory for SB model, optional for SSP and LJ.
`table_file_lammps`	Character	`''`	Filename prefix for LAMMPS tabulated potential files.

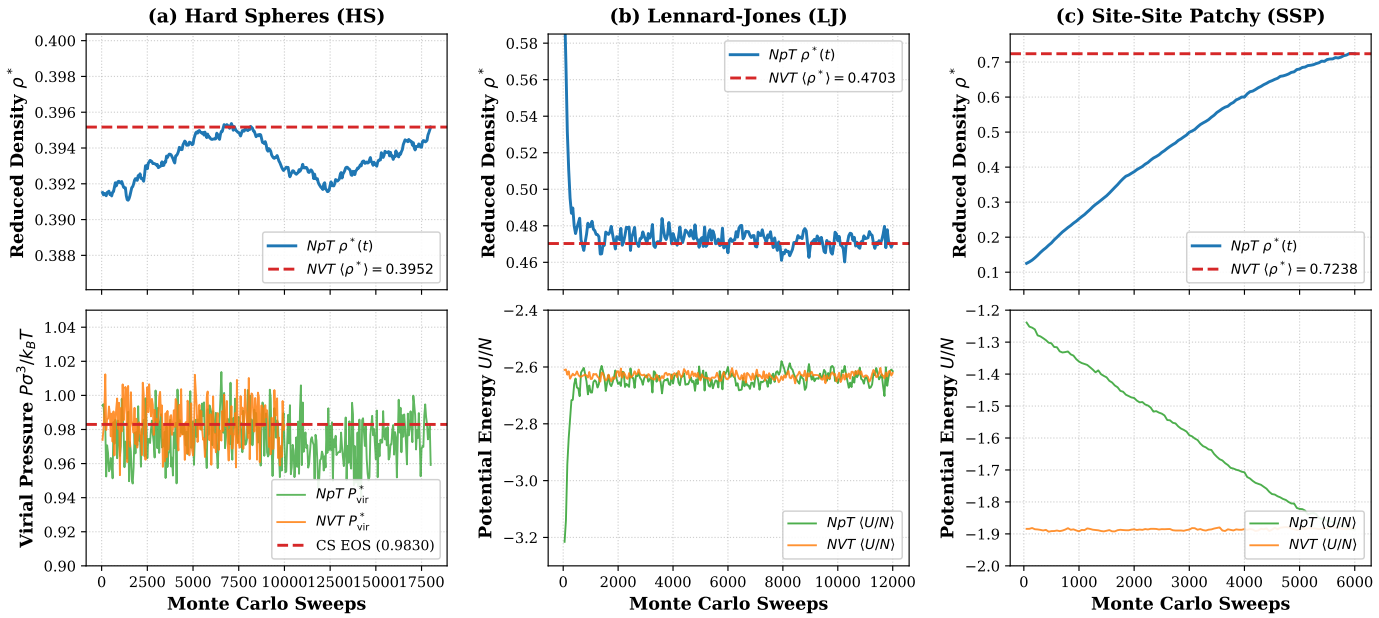


Figure 2: **Thermodynamic Consistency Benchmarks across NpT and NVT Ensembles for N = 7,695 Particles**: Time evolution of reduced density $\rho^*(t)$ (top row) and potential energy per particle $\langle U/N \rangle(t)$ or contact virial pressure $P^*(t)$ (bottom row) sharing the Monte Carlo sweep coordinate for: (a) Hard Spheres (HS) with analytical Carnahan-Starling equation-of-state verification, (b) Lennard-Jones (LJ), and (c) Site-Site Patchy (SSP) colloidal fluid, demonstrating exact statistical equivalence between ensembles.

For orientable bodies (patchy colloids, small molecules), a rotational trial move is simultaneously proposed:

$$\mathbf{q}_i' = \frac{\mathbf{q}_i + \delta\mathbf{q}}{\|\mathbf{q}_i + \delta\mathbf{q}\|} \tag{3}$$

where $\mathbf{q}_i \in \mathbb{S}^3$ is a unit quaternion and $\delta\mathbf{q}$ is sampled uniformly over an angular range controlled by parameter $o_{\max}$. The trial configuration is accepted with Metropolis probability:

$$P_{\text{acc}} = \min\left(1,\, \exp(-\beta\Delta E)\right) \tag{4}$$

where $\Delta E = E_{\text{new}} - E_{\text{old}}$ is computed directly on the GPU streaming multiprocessors.

In the isobaric-isothermal (NpT) ensemble, trial volume changes are interleaved with particle displacement sweeps at cadence N_{volf}. In isotropic mode, a volume change $\ln V \to \ln V + \delta v$ with $\delta v \in [-v_{\max}, v_{\max}]$ is proposed. The box length scales as $L' = L(V'/V)^{1/3}$, and all particle coordinates are rescaled affinely:

$$\mathbf{r}_i' = \left(\frac{V'}{V}\right)^{1/3} \mathbf{r}_i \tag{5}$$

The volume change is accepted with probability:

$$P_{\text{acc, vol}} = \min\Big(1,\, \exp\Big(-\beta\left[\Delta E_{\text{tot}} + P(V' - V)\right] + (N+1)\ln\frac{V'}{V}\Big)\Big) \tag{6}$$

where ΔE_{tot} is the total system energy difference, recomputed via a parallel GPU reduction kernel. Following an accepted volume move, the cell dimensions $L_{c,\alpha}$ are rescaled. If $L_{c,\alpha}$ drops below r_{cut}, the grid multiplicity M_α is automatically re-evaluated and the cellular neighbor lists are refreshed.

Virial Pressure for Hard Spheres. For continuous potentials, pressure is evaluated via the standard virial theorem (not implemented in the current program version). For athermal Hard Spheres (HS), where impulsive forces preclude analytic derivatives, `MCCB-gpu` calculates the instantaneous virial pressure via extrapolation of the radial distribution function $g(r)$ to contact $r = \sigma^+$:

$$P_{\text{virial}}^{\text{HS}} = \rho k_B T\left[1 + \frac{2}{3}\pi\rho\sigma^3 g(\sigma^+)\right] \tag{7}$$

During production, pairs are accumulated in fine radial histogram bins ($dr = 0.002\sigma$), and $g(\sigma^+)$ is obtained via polynomial extrapolation of a log-linear fitting all the way to contact values [28]. This log-linear fitting improves accuracy for very high packings for which the $g(r)$ slope near contact is huge.

2.3. Site-Site Patchy Interactions and Biomolecular Condensate Models

For anisotropic bodies, `MCCB-gpu` supports explicit site-site architectures where a spherical core of diameter σ is decorated with M_p surface interaction sites (patches) positioned at radial distance $r_p = \xi\sigma$ (typically $\xi = 0.5$). Each patch coordinate $\mathbf{r}_{i,\alpha}$ is updated dynamically from the particle position $\mathbf{r}_i$ and orientation quaternion $\mathbf{q}_i$ via the rotation matrix $\mathbf{R}(\mathbf{q}_i)$:

$$\mathbf{r}_{i,\alpha} = \mathbf{r}_i + \mathbf{R}(\mathbf{q}_i)\mathbf{p}_{i,\alpha}, \quad \alpha \in \{1, \ldots, M_{p,i}\} \tag{8}$$

where $\mathbf{p}_{i,\alpha}$ denotes the patch unit vector defined in the body-fixed reference frame. The total pairwise potential between particles i and j combines core-core repulsion and patch-patch attractive contributions:

$$U_{ij} = U_{\text{core}}(r_{ij}) + \sum_{\alpha=1}^{M_{p,i}}\sum_{\beta=1}^{M_{p,j}} V_{\text{pot}}(\alpha,\beta)\, U_{\text{patch}}(\|\mathbf{r}_{i,\alpha} - \mathbf{r}_{j,\beta}\|) \tag{9}$$

where $V_{\text{pot}}(\alpha,\beta) \in [0,1]$ is a discrete compatibility matrix encoding chemical patch specificity.

In addition to the cosine-squared site-site model of Palaia and Šarić (`'SSP'`) [1], `MCCB-gpu` implements the continuous square-well patchy model of Sanchez-Burgos *et al.* (`'SB'`) [2], developed to investigate liquid-liquid phase separation (LLPS) and size-conserved droplet condensation in scaffold-client protein mixtures. In this model, core-core steric repulsion is modeled by the Pseudo-Hard Sphere (PHS) potential:

$$U_{\text{PHS}}(r) = \begin{cases} C\epsilon_R\left[\left(\frac{\sigma}{r}\right)^{50} - \left(\frac{\sigma}{r}\right)^{49}\right] + \epsilon_R, & r < \frac{50}{49}\sigma \\ 0, & r \geq \frac{50}{49}\sigma \end{cases} \tag{10}$$

where $\epsilon_R = \frac{2}{3}$ is the repulsive energy scale and $C = 50\left(\frac{50}{49}\right)^{49} \approx 136.0888$, which provides a continuous approximation to hard cores that vanishes smoothly with zero derivative at $r_c^{\text{PHS}} = \frac{50}{49}\sigma \approx 1.0204\sigma$. Patch-patch attraction is governed by the Continuous Square-Well (CSW) potential:

$$U_{\text{CSW}}(d) = \begin{cases} -\frac{\epsilon_{\text{CSW}}}{2}\left[1 - \tanh\left(\frac{d - r_w}{\alpha}\right)\right], & d < r_c \\ 0, & d \geq r_c \end{cases} \tag{11}$$

where d is the patch-patch Euclidean distance, $\epsilon_{\text{CSW}} = 1/T^*$ is the attractive well depth, $r_w = 0.12\sigma$ is the well radius, $\alpha = 0.005\sigma$ controls the well edge steepness, and $r_c = 0.20\sigma$ is the interaction cutoff. In a binary mixture, multivalent scaffold proteins feature 4 tetrahedral patches with self- and cross-affinity ($V_{\text{pot}} = 1.0$), while surfactant client proteins carry 3 coplanar patches with cross-affinity toward scaffolds ($V_{\text{pot}} = 1.0$) but zero client-client patch attraction ($V_{\text{pot}} = 0.0$), capping droplet growth and preserving finite condensate sizes.

2.4. Association Volume Bias Monte Carlo (AVBMC)

In dilute colloidal and biomolecular fluids featuring strong directional attraction (such as the site-site patchy models of Palaia and Šarić [1] and Sanchez-Burgos *et al.* [2]), physical self-assembly leads to dense clusters surrounded by an ultra-dilute monomer gas. Random translation moves fail to equilibrate the cluster size distribution because the probability of an isolated monomer encountering a free cluster binding site scales as $V_{\text{bind}}/V_{\text{box}} \ll 10^{-3}$, while bound particles experience deep potential wells ($U \sim -10\, k_B T$) that prevent their vaporization in reasonable simulation times.

To eliminate this entropic/energetic bottleneck, `MCCB-gpu` incorporates Aggregation Volume Bias Monte Carlo [18, 19]. AVBMC partitions configurational space into a bonded binding region V_{bind} around cluster surfaces and the unbonded bulk volume V_{box}, proposing direct macroscopic exchanges between these domains.

Forward Association In-Move ($X \to Y$). Let X denote a state with $N_{\text{free}}(X)$ gas-phase monomers and $N_{\text{brd}}(X)$ cluster border particles:

1. A free monomer $i \in \mathcal{F}$ is selected uniformly: $P(i) = 1/N_{\text{free}}(X)$.
2. A border particle $b \in \mathcal{B}$ is chosen uniformly: $P(b) = 1/N_{\text{brd}}(X)$.
3. A target partner t is selected uniformly from the $n_{\text{neigh}}(b)$ neighbors of b: $P(t) = 1/n_{\text{neigh}}(b)$.
4. A trial position $\mathbf{r}'$ is sampled uniformly within the spherical binding volume $V_{\text{bind}} = \frac{4}{3}\pi r_{\text{bind}}^3$ centered at $\mathbf{r}_t$ ($r_{\text{bind}} = 1.5r_{\text{cut}}$): $P(\mathbf{r}') = 1/V_{\text{bind}}$.
5. To balance the bulk volume ratio, $K-1$ dummy trial positions $\{\mathbf{r}_m\}_{m=2}^{K}$ are generated uniformly in V_{box}, with trial 1 set to the current position $\mathbf{r}_i$. The bulk Rosenbluth sum is computed:

$$W_{\text{bulk}} = 1 + \sum_{m=2}^{K} \exp\left(-\beta[E(\mathbf{r}_m) - E(\mathbf{r}_i)]\right) \tag{12}$$

6. The trial cluster Boltzmann weight is $w_{\text{cluster}} = \exp\left(-\beta[E(Y) - E(X)]\right)$.

Reverse Dissociation Out-Move ($Y \to X$). In state Y, particle i resides at the cluster boundary, with $N_{\text{free}}(Y) = N_{\text{free}}(X) - 1$ and $N_{\text{brd}}(Y)$ border particles:

1. Particle i is selected from the border pool: $P(i) = 1/N_{\text{brd}}(Y)$.
2. K independent trial positions are generated uniformly in V_{box}. One trial corresponds to the original bulk location $\mathbf{r}_i$ (state X), with weight $w_{\text{rev}}(1) = \exp\left(-\beta[E(X) - E(Y)]\right) = 1/w_{\text{cluster}}$. The reverse Rosenbluth sum is:

$$W'_{\text{bulk}} = \sum_{m=1}^{K} w_{\text{rev}}(m) = \exp(\beta\Delta E) W_{\text{bulk}} \tag{13}$$

3. State X is selected with probability $w_{\text{rev}}(1)/W'_{\text{bulk}}$.

Proof of Microscopic Reversibility. Detailed balance requires $\pi(X)\alpha(X \to Y)P_{\text{acc}}(X \to Y) = \pi(Y)\alpha(Y \to X)P_{\text{acc}}(Y \to X)$, where $\pi(X) \propto e^{-\beta E(X)}$. The proposal probability densities are:

$$\alpha(X \to Y) = \frac{1}{N_{\text{free}}(X)} \frac{1}{N_{\text{brd}}(X) n_{\text{neigh}}(b)} \frac{1}{V_{\text{bind}}} \left(\frac{1}{V_{\text{box}}}\right)^{K-1} \tag{14}$$

$$\alpha(Y \to X) = \frac{1}{N_{\text{brd}}(Y)} \left(\frac{1}{V_{\text{box}}}\right)^{K} \frac{e^{\beta\Delta E}}{W'_{\text{bulk}}} \tag{15}$$

The forward and reverse acceptance probabilities implemented in `MCCB-gpu` are:

$$P_{\text{acc}}(X \to Y) = \min\left(1,\ \frac{N_{\text{brd}}(X) n_{\text{neigh}}(b) + 1}{N_{\text{free}}(X)} \frac{V_{\text{box}}}{V_{\text{bind}}} \frac{K w_{\text{cluster}}}{W_{\text{bulk}}}\right) \tag{16}$$

$$P_{\text{acc}}(Y \to X) = \min\left(1,\ \frac{N_{\text{free}}(Y) + 1}{N_{\text{brd}}(Y) n'_{\text{neigh}}} \frac{V_{\text{bind}}}{V_{\text{box}}} \frac{W'_{\text{bulk}}}{K}\right) \tag{17}$$

Letting arg_{fwd} and arg_{rev} denote the arguments of the $\min(1, \cdot)$ functions, and noting that $N_{\text{free}}(Y) + 1 = N_{\text{free}}(X)$ and $W'_{\text{bulk}} = W_{\text{bulk}}/w_{\text{cluster}}$, their product satisfies:

$$\text{arg}_{\text{fwd}} \cdot \text{arg}_{\text{rev}} \equiv 1 \tag{18}$$

Because $\text{arg}_{\text{rev}} = 1/\text{arg}_{\text{fwd}}$, it follows that:

$$\frac{P_{\text{acc}}(X \to Y)}{P_{\text{acc}}(Y \to X)} = \frac{\min(1, \text{arg}_{\text{fwd}})}{\min(1, 1/\text{arg}_{\text{fwd}})} \equiv \text{arg}_{\text{fwd}} \tag{19}$$

satisfying microscopic reversibility identically for any K, box volume V_{box}, and interaction model.

2.5. Cluster Detection via G-DBSCAN and Surface Dipole Asymmetry

A core prerequisite for AVBMC is distinguishing cluster surface particles from dense cluster interiors. Attempting to insert a monomer into the deep interior of an assembled cluster invariably results in steric overlap ($w_{\text{cluster}} = 0$).

In `MCCB-gpu`, cluster identification is carried out on the GPU using an implementation of G-DBSCAN (a GPU-accelerated algorithm for density-based spatial clustering [20]), directly adapted from the companion trajectory analysis package `trj_analysis` [4]. Two particles i and j are defined as connected if their center-to-center distance (or patch-patch distance for SSP models) satisfies $r_{ij} \le r_{\text{cl}}$. The algorithm proceeds in two stages:

1. **GPU Adjacency Graph Construction**: A CUDA kernel evaluates the neighborhood of each particle, populating compact adjacency arrays (`gneighbors`, `goffset`, `gadjacency`) on device memory. Particles with neighbor counts $z_i \ge \text{minPts} - 1$ are classified as core candidates.
2. **Parallel BFS Expansion**: Connected components are identified via a GPU breadth-first search (`gbfs`) kernel, assigning a unique `cluster_id` to all particles in each cluster.

To isolate active surface sites, each particle i in a cluster is evaluated using a local *geometric dipole asymmetry parameter*, η_i:

$$\eta_i = \frac{1}{z_i} \left\| \sum_{j \in \text{neigh}(i)} \hat{\mathbf{r}}_{ij} \right\| \tag{20}$$

where $\hat{\mathbf{r}}_{ij} = \mathbf{r}_{ij}/\|\mathbf{r}_{ij}\|$ is the unit bond vector pointing from particle i to neighbor j, and z_i is the coordination number.

- **Core Particles**: Inside a cluster, neighbors surround particle i nearly isotropically. The unit vectors cancel vectors in opposite hemispheres: $\sum_j \hat{\mathbf{r}}_{ij} \approx \mathbf{0}$, yielding $\eta_i \approx 0$.

- **Surface/Border Particles**: At the cluster boundary, neighbors reside predominantly on one side. The sum of bond vectors does not cancel, producing a large net dipole vector with $\eta_i \geq \eta_{\text{thresh}} = 0.5$.

Particles with $\eta_i \geq \eta_{\text{thresh}}$ are assigned to the border pool $\mathcal{B}$, while unbonded particles ($z_i = 0$, `cluster_id` = 0) form the free pool $\mathcal{F}$. Alternatively, an energy-based criterion ($u_i/\langle u_{\text{bulk}} \rangle \leq$ ratio) is also available via namelist control.

2.5.1. Two-Tier Dynamic AVBMC Scheduling: Nucleation vs. Condensed Phases

While G-DBSCAN and geometric dipole asymmetry η_i accurately isolate surface sites on macroscopic condensed droplets, they present two physical and computational limitations during the early stages of nucleation or when only small oligomers exist:

1. **Redundancy in Small Clusters**: For oligomers composed of a few particles ($S \leq 5$, such as dimers, trimers, and pentamers), every constituent atom resides on the boundary. Calculating the geometric dipole vector η_i or evaluating per-atom energy thresholds across all particles is computationally redundant.
2. **DBSCAN Noise Classification of Dimers**: In standard density-based clustering, nodes require a minimum coordination $z_i \geq \text{minPts}-1$ to serve as core seeds. For the conventional setting minPts = 3, dimers ($z_i = 1$) and linear trimers are classified as unclustered noise (`cluster_id` = 0). Consequently, unassociated particles cannot be targeted for association moves onto dimers, impeding the initial nucleation transition.

To resolve this limitation, `MCCB-gpu` implements a dynamic two-tier scheduling architecture that automatically switches between two operational regimes based on the instantaneous maximum cluster size $S_{\max}$:

Tier 1: Pre-Nucleation / Small-Cluster Regime ($S_{\max} \leq S_{thresh}$). When all clusters in the system contain $S \leq S_{\text{thresh}}$ particles (configured via `cluster_threshold`, default 5):

- Full host DBSCAN BFS traversal, cluster size distribution analysis, trajectory logging (`mclast_clconf.lammpstrj`), and surface dipole asymmetry evaluations are completely bypassed.
- Particles with cluster mask active are classified using fast $O(N)$ GPU cell-list neighbor counting (`neighbor_search`) into:

$$\text{Bound Pool } \mathcal{B} : \{i \mid z_i \geq z_{\text{bound}}\} \tag{21}$$

$$\text{Isolated Pool } \mathcal{F} : \{i \mid z_i = 0\} \tag{22}$$

  where z_{bound} (namelist `bound_min_neighbors`, default 1) ensures all bonded oligomers (including dimers with $z_i = 1$) constitute the target binding pool $\mathcal{B}$, while unbonded monomers constitute the bulk pool $\mathcal{F}$.
- Lightweight AVBMC exchange attempts are executed frequently, once every n_{avbmc} sweeps (configured via `n_avbmc`, default 10), accelerating monomer association into dimers and oligomers with sub-millisecond execution overhead.
- A full DBSCAN cluster search is performed periodically once every $n_{\text{cluster_init}}$ sweeps (namelist `ncluster_init`, default 500) to monitor whether a critical nucleus has formed.

Tier 2: Post-Nucleation / Condensed Regime ($S_{\max} > S_{thresh}$). As soon as a nucleus exceeding S_{thresh} particles is detected during a periodic check, the simulation triggers an automatic regime switch (`cluster_regime_large = .true.`):

- The effective AVBMC frequency is dynamically linked to the full cluster analysis period, setting $n_{\text{avbmc}} = n_{\text{cluster}}$.
- Every n_{cluster} sweeps (namelist `ncluster`), G-DBSCAN connected component identification, geometric dipole surface detection ($\eta_i \geq \eta_{\text{thresh}}$), and boundary logging (`mclast_brdconf.lammpstrj`) are activated, targeting trial association moves exclusively to the physical interface of the growing cluster.
- **Dynamic Reversibility**: If the cluster subsequently evaporates or dissolves ($S_{\max} \leq S_{\text{thresh}}$), `MCCB-gpu` automatically reverts to Tier 1, restoring lightweight high-frequency sampling without user intervention.

2.6. Multicomponent Particle Identity Swaps ($A \leftrightarrow B$)

In dense multicomponent mixtures, such as non-additive hard sphere (NAHS) fluids and binary glass formers, structural relaxation is arrested by compositional jamming: particles cannot exchange positions because intermediate steric barriers prevent physical diffusion. Identity swap moves ($A \leftrightarrow B$) exchange the species labels of two distinct particles while keeping their spatial coordinates fixed, bypassing free-energy barriers along compositional degrees of freedom.

`MCCB-gpu` implements two complementary GPU checkerboard swap kernels:

1. **Intra-Cell Swaps**: Within an active checkerboard cell C, two distinct particles $i, j \in C$ of different species ($S_i \neq S_j$) are chosen uniformly at random with probability:

$$P_{\text{pair}}(i, j \mid C) = \frac{2}{n_C(n_C - 1)} \tag{23}$$

   where n_C is the particle occupancy of cell C.
2. **Cross-Cell Disjoint Swaps**: Two mutually non-adjacent active cells C_1, C_2 separated by $\geq r_{\text{cut}}$ are selected, and particles $i \in C_1$, $j \in C_2$ with $S_i \neq S_j$ are selected with probability $1/(n_{C_1} n_{C_2})$. A symmetric random choice determines the swap direction.

Proof of Microscopic Reversibility. Because particle coordinates are unchanged by identity swaps, the cell occupancies n_C are strictly invariant:

$$n_C(Y) = n_C(X) \implies \alpha_{\text{prop}}(Y \to X) \equiv \alpha_{\text{prop}}(X \to Y) \tag{24}$$

The proposal ratio $\alpha(Y \to X)/\alpha(X \to Y)$ is identically unity. For athermal hard spheres, the move is accepted if and only if no hard-core overlaps are created. For continuous and patchy potentials, the energy difference is:

$$\begin{aligned}\Delta E = &\sum_{k\neq i,j} \Big[U(S_j, S_k, \mathbf{r}_{ik}) - U(S_i, S_k, \mathbf{r}_{ik})\Big] \\ &+ \sum_{k\neq i,j} \Big[U(S_i, S_k, \mathbf{r}_{jk}) - U(S_j, S_k, \mathbf{r}_{jk})\Big] + \Delta E_{ab}\end{aligned} \tag{25}$$

where $\Delta E_{ab} = U(S_j, S_i, \mathbf{r}_{ij}) - U(S_i, S_j, \mathbf{r}_{ij}) \equiv 0$ for symmetric pair interactions. The move is accepted with standard Metropolis probability $\min(1, e^{-\beta\Delta E})$.

Generalization to Multicomponent Mixtures ($N_{species} \geq 2$). In systems with $N_{\text{species}} > 2$, the two species labels (A, B) are chosen uniformly at random from all available pairs of distinct species for each swap attempt. This preserves microscopic reversibility across arbitrary ternary, quaternary, and polydisperse mixtures.

2.7. Persistent In-Memory Hybrid Monte Carlo (HMC)

To incorporate collective conformational rearrangements, `MCCB-gpu` provides a Hybrid Monte Carlo (HMC) module [29, 30] that periodically interleaves canonical MC sweeps with microcanonical (NVE) Molecular Dynamics (MD) trajectory segments generated by the embedded LAMMPS engine [3].

Explicitly, starting from an initial configuration $\mathbf{q}_0$, particle linear and angular velocities $\mathbf{p}_0$ are sampled from the Maxwell-Boltzmann distribution at temperature T. A microcanonical MD trajectory of N_{md} timesteps of duration Δt is integrated via the velocity Verlet algorithm with rigid-body quaternion dynamics (`fix rigid/nve molecule`). The proposed state $(\mathbf{q}_\tau, \mathbf{p}_\tau)$ is accepted with generalized Metropolis probability:

$$P_{\text{acc}}^{\text{HMC}} = \min\left(1,\, \exp\left(-\beta[\mathcal{H}(\mathbf{q}_\tau, \mathbf{p}_\tau) - \mathcal{H}(\mathbf{q}_0, \mathbf{p}_0)]\right)\right) \tag{26}$$

where $\mathcal{H}(\mathbf{q}, \mathbf{p}) = U(\mathbf{q}) + \sum_i \frac{\mathbf{p}_i^2}{2m_i}$ is the total classical Hamiltonian. If rejected, the trial trajectory is discarded and the system reverts to configuration $\mathbf{q}_0$.

Analytical vs. Tabulated Forces in LAMMPS. A crucial technical consideration when interfacing Monte Carlo with LAMMPS MD is whether pairwise forces in the MD stage are evaluated analytically or via numerical interpolation tables:

- **Analytical Evaluation** (`table_lammps = .false.`): When standard LAMMPS possesses native built-in pair styles for the underlying Hamiltonian, HMC executes purely analytically without tables. This is the case for Lennard-Jones fluids (`pair_style lj/cut`) and the Palaia SSP site-site patchy model (`pair_style hybrid/overlay lj/cut cosine/squared`), where exact analytical forces are computed for both core repulsion and patch-patch attraction.
- **Tabulated Evaluation** (`table_lammps = .true.`): When the interaction model involves functional forms not natively implemented in standard LAMMPS, *LAMMPS necessarily operates via numerical tables* (`pair_style table linear`). In particular, for the Sanchez-Burgos (SB) patchy model, the Continuous Square-Well patch interaction Eq. (11) is modulated by a hyperbolic tangent (tanh) smoothing function that lacks a native analytical pair style in LAMMPS. Consequently, LAMMPS must evaluate SB patch forces via tabulated interpolation (typically using 10^5 grid points to accurately resolve the steep well boundary).
- **Hamiltonian Consistency**: While `MCCB-gpu` evaluates the SB model analytically in pure MC sweeps, enabling `table_mc = .true.` in conjunction with `table_lammps = .true.` during HMC guarantees exact numerical equivalence between the MC and MD Hamiltonians, preventing acceptance degradation from slight interpolation offsets.
- **Short-Range / Long-Range Potential Splitting**: Tabulated potentials for the MC stage can be stored in separate files from those of the LAMMPS stage. This allows delegating smooth, long-range tails to the MD stage (which would otherwise require large checkerboard cells $L_{c,\alpha} \geq r_{\text{cut}}$ and inhibit MC concurrency), while retaining the full efficiency of the GPU checkerboard engine for the short-range, directional associative bonding.

Elimination of Disk and Task Forking Overheads. Conventional implementations interface MC codes with LAMMPS by writing ASCII data files (`lammps_hmc.data`) to disk and invoking an external binary or calling `clear` between trials. As quantified in Section 5.3 and Figure 4, disk serialization and lexical parsing consume 122.9 ms per trial, while issuing `clear` incurs a 351.5 ms penalty to rebuild neighbor lists and reallocate GPU memory buffers.

`MCCB-gpu` resolves both bottlenecks by implementing a *persistent in-memory architecture*:

1. The LAMMPS instance, box topology, pair potentials, and GPU neighbor lists are initialized once during the first trial and maintained in host memory throughout the simulation.
2. Coordinates are injected directly into LAMMPS via `lammps_scatter_atoms("x")` and extracted via `lammps_gather_atoms("x")`, completing in 0.43 ms (287× faster than disk I/O).
3. For patchy particles, rigid bodies are projected onto central core atoms and satellite patch sites. Before each MD segment, periodic boundary image flags are zeroed (`set group all image 0 0 0`) and `fix rigid/nve molecule` is dynamically refreshed to prevent coordinate unwrap drift across successive HMC trials.

3. Software Architecture and Program Description

3.1. Modular Structure and Source Organization

`MCCB-gpu` is structured into modular compilation units utilizing CUDA Fortran and Fortran 2008 with standard ISO C bindings:

- `src/Main.cuf`: Main simulation driver, command-line parsing, initialization, time evolution loop, periodic checkpointing, and clean shutdown.
- `src/Subsweep_Energy_CUDA.cuf`: Core GPU computational engine containing the checkerboard sub-sweep kernels (`subsweep_HS`, `subsweep_LJ`, `subsweep_angular`, `subsweep_sitesite`), identity swap kernels, and energy reduction routines.
- `src/Cell.cuf` & `src/cuda_fortran.cuf`: Host and device data structures for 3D cellular grid indexing, particle coordinate packing, and GPU device memory allocations.
- `src/mod_clusters.cuf`: G-DBSCAN GPU neighbor search, parallel BFS traversal kernel (`gbfs`), and geometric dipole surface asymmetry calculation.
- `src/mod_avbmc.cuf`: AVBMC trial generator, Rosenbluth sum computation, and acceptance evaluation.
- `src/lammps_hmc_wrapper.cuf`: C-API interface to `liblammps`, persistent memory management, coordinate scatter/gather, and rigid-body quaternion projection.
- `src/potential_functions.cuf`: Analytical pair interaction routines for continuous isotropic and site-site patchy models (`vphs`, `vcsw`, `vlj_palaia`) equipped with dual host and device attributes.
- `src/Read_input_data_nml.cuf`: Namelist parsing, input verification, and restart checkpoint writing (`input-restart.nml`).
- `src/netcdf_io.f90`: High-performance binary trajectory writer implementing the AMBER NetCDF convention (delivering 35× file compression over text formats) also available in LAMMPS dumps.

3.2. Input Variables and Namelists

Simulation parameters are provided via standard Fortran namelists in a single input file (e.g., `datos.nml`). Tables 1–4 document all input variables, allowable values, and physical meanings.

3.3. Input and Output Files

Input Files.

- `datos.nml`: Primary input file containing namelists (`&Control_Params`, `&MC_Params`, `&Potential_Params`, `&Lammps_Params`) followed by pairwise interaction matrices (σ_{ij}, ε_{ij}, $r_{c,ij}$).
- `data.atoms`: Initial atomic configuration formatted according to the standard LAMMPS data specification, specifying box bounds, masses, and atomic coordinates (and quaternions for patchy models).
- `pot*.dat`: User-tabulated pairwise potential files (for `model = 'TABLE'`).

Output Files.

- `run-data.dat`: Time series of thermodynamic observables: Step, Temperature, Density, Volume, L_x, L_y, L_z, and Potential Energy per atom.
- `trajectory.nc`: High-density binary NetCDF trajectory containing per-frame coordinates, quaternions, cell metrics, and metadata.
- `data.restart` & `input-restart.nml`: Pair of exact checkpoint files capturing the full microstate and active simulation parameters, enabling seamless restarts without data loss.
- `clusevol_mc.dat`: Time series of cluster statistics: Step, Cluster Count N_{clust}, Maximum Cluster Size $S_{\max}$, Clustered Particle Count, and Clustered Fraction (%).
- `mclast_conf.lammpstrj`: Final configuration formatted in LAMMPS trajectory format for visual analysis via VMD or OVITO.

4. Verification and Algorithmic Validation

Codes and reports for the verification tests listed below are included with the software.

4.1. Thermodynamic Consistency ($NpT \leftrightarrow NVT$)

To verify the reliability of the code, in addition to checks with literature data and other in-house codes, we perform a two-stage workflow as follows:

1. **Stage 1** (NpT): An isobaric-isothermal simulation is equilibrated at target pressure P^*_{target} and temperature T^*, recording the mean volume $\langle V \rangle_{NpT}$ (density $\langle \rho^* \rangle_{NpT}$) and generating an exact checkpoint (`data.restart`).
2. **Stage 2** (NVT): A canonical simulation is executed from the restart configuration at fixed density $\rho^* = \langle \rho^* \rangle_{NpT}$.
3. **Validation**: Equivalence of internal potential energy $\langle U/N \rangle_{NVT} \approx \langle U/N \rangle_{NpT}$ is verified for continuous models, and virial pressure $\langle P_{\text{virial}} \rangle$ is cross-validated against the analytical Carnahan-Starling equation of state for Hard Spheres.

As summarized in Table 5 and Figure 2:

- For **Hard Spheres** ($T^* = 1.0, P^* = 1.0$), the contact virial pressure reproduces the Carnahan-Starling equation of state [7]:

$$P_{\text{CS}}(\eta) = \rho k_B T \frac{1 + \eta + \eta^2 - \eta^3}{(1-\eta)^3}, \quad \eta = \frac{\pi}{6}\rho\sigma^3 \tag{27}$$

Table 5: **Thermodynamic Consistency Validation across Interaction Models**

Model	Observable	NpT Ensemble	NVT Ensemble	Discrepancy / Agreement
Lennard-Jones	Density $\langle\rho^*\rangle$	0.4723 ± 0.0041	0.4703	0.43%
($T^* = 1.5, P^* = 0.6$)	Energy $\langle U/N\rangle$	-2.640 ± 0.023	-2.631 ± 0.011	**0.34%** (0.35σ)
Hard Spheres	Density $\langle\rho^*\rangle$	0.3936 ± 0.0010	0.3952	0.39%
($T^* = 1.0, P^* = 1.0$)	Virial P^*_{NpT}	0.9719 ± 0.0010	—	**0.36%** vs $P_{\mathrm{CS}} = 0.9754$
	Virial P^*_{NVT}	—	0.9831 ± 0.0010	**0.02%** vs $P_{\mathrm{CS}} = 0.9830$
Site-Site Patchy	Density $\langle\rho^*\rangle$	0.7158 ± 0.0063	0.7238	1.11%
($T^* = 0.154, P^* = 0.05$)	Energy $\langle U/N\rangle$	-1.873 ± 0.010	-1.885 ± 0.003	**0.64%** (1.15σ)
	Restart U/N	-1.88562	-1.88560 ± 0.0003	**0.001%** (Exact Match)

within **0.36**% in NpT (0.9719 vs 0.9754) and **0.02**% in NVT (0.9831 vs 0.9830), with an inter-ensemble virial discrepancy of only 1.16%.

- For **Lennard-Jones**, internal energies match with a discrepancy of only 0.34% (Z-score of 0.35σ).
- For **Site-Site Patchy colloids**, internal energies match within 0.64% ($Z = 1.15\sigma$), with exact configuration restart matching to within 0.001%.
- For **Biomolecular Condensates (Sanchez-Burgos Model)**, the implementation was cross-validated against an independent benchmark simulated in LAMMPS [3] for a binary 50:50 mixture of 2,000 scaffold and client proteins ($N = 2{,}000, \rho^* = 0.136, T^* = 0.09$). The initial total potential energy evaluated by `MCCB-gpu` (−151.70959) matches the LAMMPS benchmark (−151.70991) and the analytical Hamiltonian (−151.71093) with a relative discrepancy below 0.0002%, confirming exact consistency across CPU and GPU checkerboard implementations.

4.2. *Machine-Precision Numerical Verification of Detailed Balance*

To rigorously confirm that theoretical reversibility proofs translate to error-free numerical implementations, independent automated test harnesses were executed:

1. **AVBMC Invariant**: In an independent test harness evaluating 5,000 state transitions across Palaia patchy colloidal clusters, the flux reversibility invariant $|\mathrm{arg}_{\mathrm{fwd}} \cdot \mathrm{arg}_{\mathrm{rev}} - 1.0|$ Eq. (18) yielded a mean deviation of $\mathbf{3.17 \times 10^{-17}}$ and a maximum deviation of $\mathbf{4.44 \times 10^{-16}}$, matching IEEE 754 double precision machine epsilon ($\sim 2.22 \times 10^{-16}$).
2. **Identity Swaps**: Tested across 11,717 intra-cell and cross-cell candidate transitions in dense non-additive hard sphere and continuous mixtures, the detailed balance residual $\delta_{\mathrm{DB}} = |\pi(X)T(X \to Y) - \pi(Y)T(Y \to X)|$ satisfied $\delta_{\mathrm{DB}} < \mathbf{10^{-15}}$ with a 100.0% pass rate.

5. Performance Benchmarks and Discussion

5.1. *CPU vs. GPU Scaling: Normalizing Moves per Particle*

We benchmarked `MCCB-gpu` (v2.8.0) on an **NVIDIA RTX PRO 4500 Blackwell GPU** (32 GB VRAM) against an optimized serial CPU code **gpMC** [31] (compiled with Intel Fortran `ifx` 2025.2.0) on a 32-core **Intel Xeon Gold 6548Y+ CPU** (2.20 GHz) for a binary Lennard-Jones mixture (N_1:N_2 = 2:1, $T^* = 3.0$, $r_c = 2.5\sigma$) scaling from $N = 8{,}000$ to 125,000 particles for 100 MC sweeps. Larger sample sizes were untreatable for the serial code.

Normalization of Monte Carlo sweeps. A critical methodological distinction between serial CPU code and checkerboard GPU codes lies in the operational definition of a Monte Carlo sweep:

- In **gpMC**, each sweep evaluates exactly 1 trial displacement per particle ($\mu_{\mathrm{CPU}} = 1.0$).
- In **MCCB-gpu**, the checkerboard engine evaluates $N_{\mathrm{move}} = 125$ trial moves per active cell across the 8 sublattices, attempting $N_{\mathrm{trans}} = N_{\mathrm{cells}} \times N_{\mathrm{move}}$ moves per sweep. The number of moves attempted *per particle per sweep* is:

$$\mu_{\mathrm{GPU}} = \frac{N_{\mathrm{cells}} \times N_{\mathrm{move}}}{N} \tag{28}$$

yielding $\mu_{\mathrm{GPU}} = 3.375, 1.728, 1.953,$ and 2.744 moves/particle/sweep for $N = 8{,}000, 15{,}625, 32{,}768,$ and 125,000, respectively.

To provide an unbiased comparison of actual sampling throughput, we report both the raw measured wallclock time t_{raw} and the normalized GPU runtime $t_{\mathrm{norm}} = t_{\mathrm{raw}}/\mu_{\mathrm{GPU}}$, along with the effective sampling speedup:

$$S_{\mathrm{eff}} = \frac{t_{\mathrm{CPU}}}{t_{\mathrm{norm,\,GPU}}} = S_{\mathrm{raw}} \times \mu_{\mathrm{GPU}} \tag{29}$$

As detailed in Table 6 and Figure 3:

- **Peak Speedup**: At $N = 125{,}000$, `MCCB-gpu` achieves raw speedups of 22.4× (NVT) and 25.1× (NpT). Accounting for moves per particle, the true sampling speedup reaches **61.4**× in NVT and **68.8**× in NpT.
- **Anomalous GPU Saturation**: Between $N = 15{,}625$ and $N = 125{,}000$, GPU execution time remains essentially flat on the RTX PRO 4500 (8.73 → 7.86 s in NVT, 11.88 → 10.14 s in NpT) despite an 8× increase in particle count! This plateau demonstrates peak CUDA warp occupancy and block memory coalescing on modern GPU architectures.

Table 6: **Simulation Runtime (Seconds) and Speedup across CPU and GPU Architectures (*NVT* and *NpT* Ensembles) for a binary Lennard-Jones mixture (N_1:N_2 = 2:1, T^* = 3.0, r_c = 2.5σ)**

	Moves/Part.	**gpMC CPU Time (s)**		**MCCB-gpu GPU Time (s)**		**Effective Speedup**	
Particles (N)	(μ_{GPU} / μ_{CPU})	*NVT*	*NpT*	*NVT* **(Norm)**	*NpT* **(Norm)**	*NVT*	*NpT*
8000	3.375 / 1.0	9.95	15.16	4.97 (1.47)	6.26 (1.85)	**6.76**× (2.00×)	**8.17**× (2.42×)
15 625	1.728 / 1.0	23.59	35.95	8.73 (5.05)	11.88 (6.88)	**4.67**× (2.70×)	**5.23**× (3.03×)
32 768	1.953 / 1.0	38.63	59.23	8.05 (4.12)	9.94 (5.09)	**9.37**× (4.80×)	**11.64**× (5.96×)
125 000	2.744 / 1.0	175.91	254.09	7.86 (2.86)	10.14 (3.70)	**61.41**× (22.39×)	**68.76**× (25.06×)

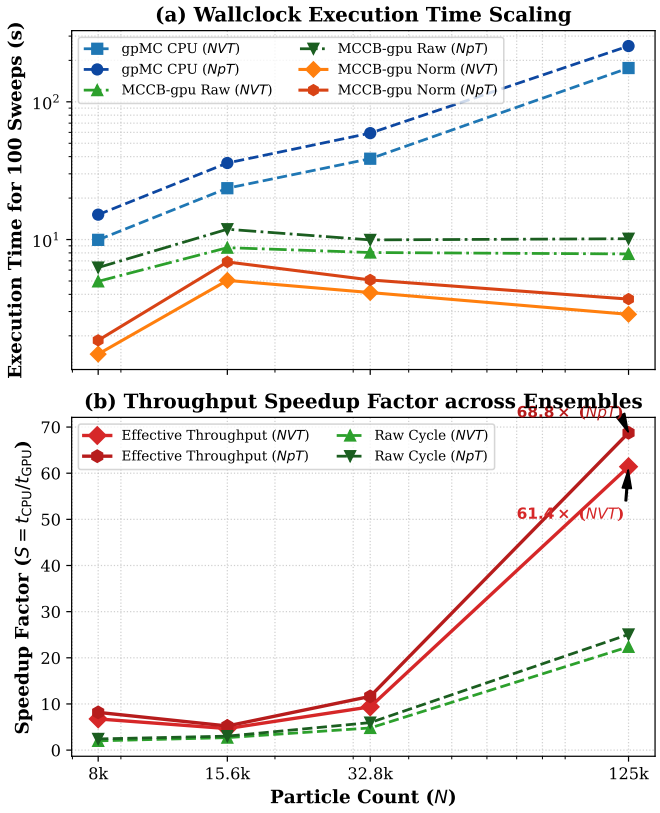


Figure 3: **gpMC (CPU) vs. MCCB-gpu (GPU) Performance Scaling across *NVT* and *NpT* Ensembles**: (a) Wallclock execution times for 100 sweeps on Intel Xeon Gold 6548Y+ vs. NVIDIA RTX PRO 4500. (b) Effective throughput speedup factor S_{eff} accounting for moves per particle, reaching 61.4× in *NVT* and 68.8× in *NpT*.

5.2. Large-Scale Scaling: LAMMPS vs. MCCB-gpu

Table 7 evaluates scaling up to nearly four million particles ($N \approx 4 \times 10^6$) comparing `MCCB-gpu` against GPU-accelerated LAMMPS on NVIDIA RTX 2000 and RTX 4500 hardware for a binary Lennard-Jones mixture (N_1:N_2 = 2:1, T^* = 3.0, r_c = 2.5σ). `MCCB-gpu` maintains linear $O(N)$ scaling, executing 100 sweeps for 3.94×10^6 particles in just 25.6 s on the RTX 4500.

Table 7: **Large-Scale Simulation Runtimes (Seconds for 100 Steps) for a binary Lennard-Jones mixture** (N_1:N_2 = 2:1, T^* = 3.0, r_c = 2.5σ)

	LAMMPS Time (s)		**MCCB-gpu Time (s)**	
Particles (N)	**RTX 2000**	**RTX 4500**	**RTX 2000**	**RTX 4500**
492 450	7.0	5.0	18.0	3.8
1 969 920	22.0	18.0	66.0	13.2
3 939 840	44.0	34.0	123.0	25.6

5.3. In-Memory Hybrid Monte Carlo Overhead Breakdown

Profiling the HMC coupling layer for N = 7,695 tetrahedral patchy colloids (38,475 explicit interaction sites) reveals:

- **Data Transfer Acceleration**: Writing and parsing ASCII data files to disk consumes 122.88 ms per trial (96.7% of baseline transfer time). Direct in-memory pointer transfers via `scatter_atoms` and `gather_atoms` require only 0.43 ms—a **287**× acceleration.
- **Persistent State Optimization**: Eliminating in-process `clear` calls saves 351.5 ms of neighbor list and memory allocation overhead per trial, delivering a 1.66× overall speedup.
- **Crossover Length**: For GPU-accelerated tabulated potentials, physical MD integration begins to dominate over coupling overhead at $N^*_{\text{md}} \approx 20$ timesteps.

5.4. Sampling Acceleration: AVBMC and Identity Swaps

AVBMC on Patchy Colloidal Clusters. As shown in Figures 5 and 6, in a dilute binary patchy colloidal fluid (N = 7,695, ρ = 0.001218, T^* = 0.154), standard canonical MC undergoes kinetic arrest: the maximum cluster size stagnates at $S_{\text{max}} \approx$ 1068, and the clustered fraction declines as monomers evaporate into the void.

With AVBMC enabled, the association acceptance rate reaches $P_{\text{AV,in}}$ = **55.6**% (Figure 5b), transferring over 440

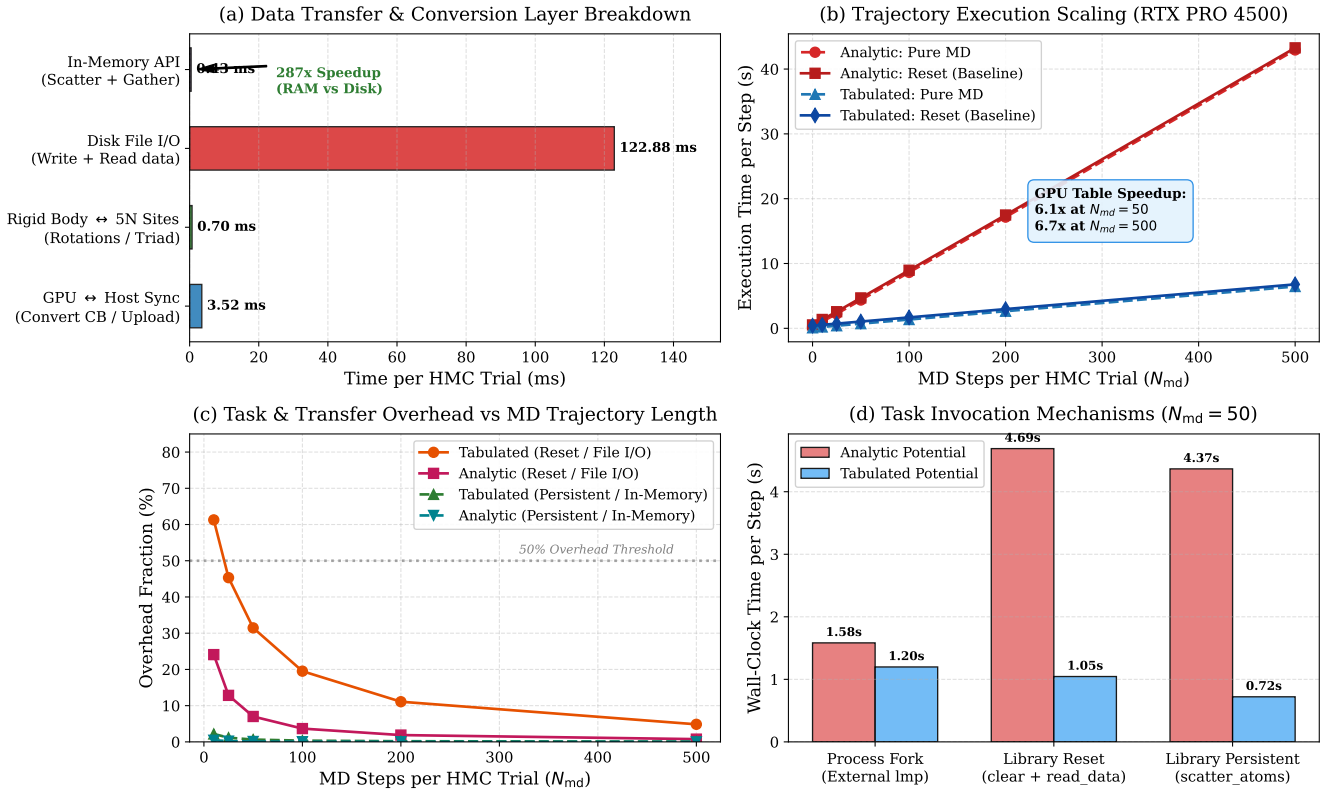


Figure 4: **Profiling of In-Memory HMC Coupling with LAMMPS on NVIDIA RTX PRO 4500**: (a) Latency breakdown showing the 287× speedup of direct in-memory C-API coordinate transfer (0.43 ms) over disk ASCII file I/O (122.9 ms). (b) MD trajectory propagation scaling with N_{md}, highlighting a 6.1×–6.7× throughput advantage for GPU-accelerated tabular potentials over analytic overlays. (c) Coupling overhead fraction versus N_{md}. (d) Comparison of task invocation paradigms for $N_{\mathrm{md}} = 50$ steps.

monomers directly to active cluster surface sites over 200 sweeps and driving cluster growth to $S_{\max} = 1374$ (Figure 5a). As evidenced by the cluster size distribution in Figure 6, AVBMC actively populates the tail of large assemblies ($s > 500$). Remarkably, the computational overhead of performing G-DBSCAN clustering and 800 AVBMC trial moves every 5 sweeps is under **2.8**% (473.7 ms/sweep vs 460.5 ms/sweep).

Identity Swaps in Jammed Non-Additive Mixtures. In a dense non-additive hard sphere mixture ($N = 2{,}916$, $P^* = 70.24$), canonical simulations exhibit completely frozen compositional sampling: the species autocorrelation function defined as

$$C_s(\Delta t) = \frac{\langle (s_i(t_0 + \Delta t) - \bar{s})(s_i(t_0) - \bar{s}) \rangle}{\mathrm{Var}(s)} \tag{30}$$

where $s_i(t) \in \{+1, -1\}$ denotes the species identity of particle i at time t, and the average $\langle \dots \rangle$ is taken over all N particles and time origins t_0, remains $C_s(\Delta t) \equiv 1$ and zero particles exchange chemical identities. With GPU identity swaps enabled, 41.4% of particles exchange species labels, rapidly decaying $C_s(\Delta t)$ to 0.45. This dramatic ergodicity restoration introduces only +1.02 ms/sweep (+18.6%) in GPU latency across more than 71 million attempted swap moves.

6. Conclusions

We have introduced `MCCB-gpu`, an open-source, massively parallel GPU Monte Carlo simulation code for soft-matter and molecular modeling. By combining a 3D checkerboard domain decomposition with three targeted sampling algorithms—AVBMC guided by geometric dipole surface asymmetry, multicomponent particle identity swaps, and persistent in-memory Hybrid Monte Carlo/LAMMPS—`MCCB-gpu` bridges the gap between massive GPU parallel efficiency and advanced free-energy sampling.

The tests confirm the thermodynamic consistency between NpT and NVT ensembles. Accuracy for hard spheres has been tested against analytical EoS ($< 0.04\%$). Additive and non-additive hard sphere mixtures were tested against the results of the DynamO hard-core event-driven simulation engine [32]. Benchmarking against an optimized serial CPU Monte Carlo code demonstrates algorithmic sampling throughput speedups up to **68.8**× on modern NVIDIA GPUs. Testing against LAMMPS results for site-site patchy models and simple mixtures, both using analytical and tabulated potentials, confirms the reliability of the results. The source code, complete documentation, automated test suites, and tutorials are freely available under the GPLv3 license at `https://github.com/adpozuelo/MC_Checkerboard`.

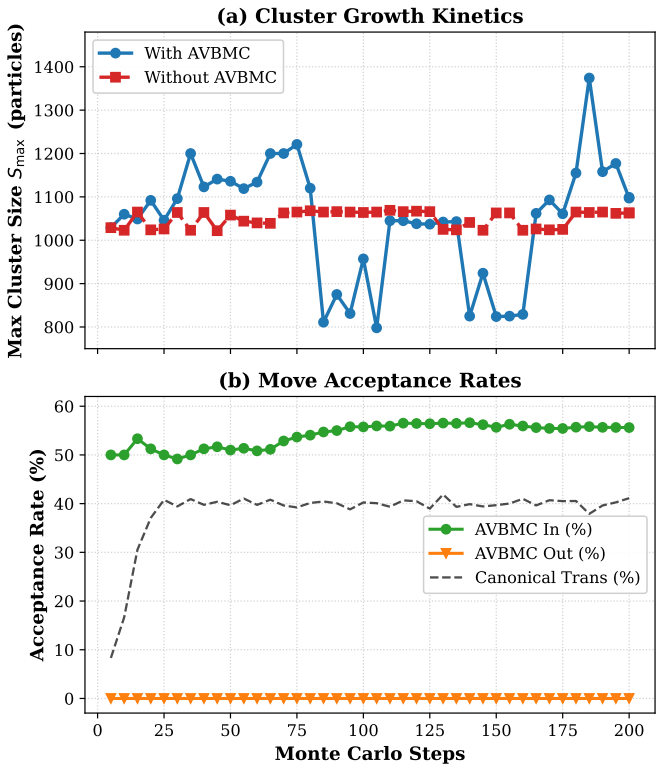


Figure 5: **Comparative Benchmark of Patchy Colloidal Cluster Sampling With vs. Without AVBMC**: (a) Maximum cluster size evolution $S_{\max}(t)$ demonstrating accelerated aggregation kinetics and overcoming kinetic traps. (b) Monte Carlo acceptance rates for association ($P_{\text{AV,in}}$), dissociation ($P_{\text{AV,out}}$), and standard canonical displacement (P_{trans}), sharing the Monte Carlo simulation steps coordinate.

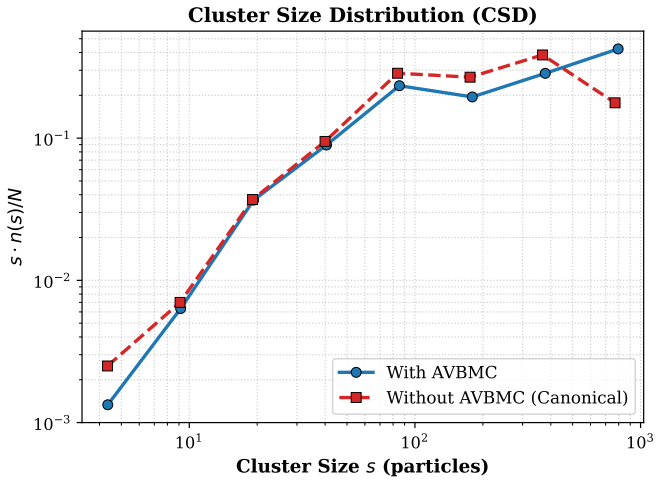


Figure 6: **Final Cluster Size Distribution (CSD) for Patchy Colloids With vs. Without AVBMC**: Normalized probability $P(s)$ versus cluster size s after 200 sweeps, highlighting that AVBMC significantly accelerates aggregation into large macromolecular clusters ($s > 500$) compared to kinetically arrested canonical sampling.

7. Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Gemini 3.8 in order to generate input parameter and output file tables, performance checks and figures, the latter under the strict guidance of the authors. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. During revision, the authors also used OpenAI Codex to compare the repository changes with the manuscript and assist with updating the text, equations and parameter tables. During the implementation of the software, the authors have used OpenAI Codex to assist in the design of the algorithm for hard sphere simulations, Gemini 3.8 to refactor and document the code, and to implement the AVBMC and identity swap algorithms. These code implementations were double-checked for thermodynamic internal consistency, and against Molecular Dynamics results from LAMMPS for continuous potentials and DynamO for hard sphere mixtures [32].

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MICIU/AEI/10.13039/501100011033) and by "ERDF A way of making Europe" through Grant PID2023-151751NB-I00. Computational resources were provided by the high-performance computing clusters at the Instituto de Química Física Blas Cabrera (IQF-CSIC) and the Galicia Supercomputing Center (CESGA).

References

[1] I. Palaia, A. Šarić, Controlling cluster size in 2D phase-separating binary mixtures with specific interactions, J. Chem. Phys. 156 (19) (2022) 194902. doi:10.1063/5.0087769.

[2] I. Sanchez-Burgos, J. A. Joseph, R. Collepardo-Guevara, J. R. Espinosa, Size conservation emerges spontaneously in biomolecular condensates formed by scaffolds and surfactant clients, Sci. Rep. 11 (1) (2021) 15241. doi:10.1038/s41598-021-94309-y.

[3] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Comput. Phys. Commun. 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.

[4] A. Díaz-Pozuelo, R. Lomba, E. Lomba, trj_analysis: High-Performance GPU-Accelerated Trajectory Analysis and Graph-Mining Package for Complex Colloidal Assemblies, Available at: https://github.com/adpozuelo/trj_analysis (2026).

[5] D. Frenkel, B. Smit, Understanding Molecular Simulation: From Algorithms to Applications, 2nd Edition, Academic Press, San Diego, 2001.

[6] M. P. Allen, D. J. Tildesley, Computer Simulation of Liquids, 2nd Edition, Oxford University Press, Oxford, 2017.

[7] N. F. Carnahan, K. E. Starling, Equation of state for nonattracting rigid spheres, J. Chem. Phys. 51 (2) (1969) 635–636. doi:10.1063/1.1672048.

[8] E. Bianchi, J. Largo, P. Tartaglia, E. Zaccarelli, F. Sciortino, Phase diagram of patchy colloids: towards empty liquids, Phys. Rev. Lett. 97 (16) (2006) 168301. doi:10.1103/PhysRevLett.97.168301.

[9] F. Sciortino, E. Bianchi, J. F. Douglas, P. Tartaglia, Self-assembly of patchy particles into polymer networks: A molecular dynamics study, J. Chem. Phys. 126 (19) (2007) 194903. doi:10.1063/1.2730797.

[10] E. G. Noya, I. Kolovos, G. Doppelbauer, G. Kahl, E. Bianchi, Phase diagram of inverse patchy colloids assembling into an equilibrium laminar phase, Soft Matter 10 (42) (2014) 8464–8474. doi:10.1039/c4sm01559b.

[11] A. Ninarello, L. Berthier, D. Coslovich, Models and algorithms for the next generation of glass transition studies, Phys. Rev. X 7 (2) (2017) 021039. doi:10.1103/PhysRevX.7.021039.

[12] L. Berthier, P. Charbonneau, D. Coslovich, A. Ninarello, M. Ozawa, S. Yaida, Configurational entropy measurements in extremely supercooled liquids that break the glass ceiling, Proc. Natl. Acad. Sci. U.S.A. 114 (43) (2017) 11356–11361. doi:10.1073/pnas.1706860114.

[13] J. A. Anderson, C. D. Lorenz, A. Travesset, General purpose molecular dynamics simulations fully implemented on graphics processing units, J. Comput. Phys. 227 (10) (2008) 5342–5359. doi:10.1016/j.jcp.2008.01.047.

[14] S. Whitelam, P. L. Geissler, Avoiding unphysical kinetic traps in Monte Carlo simulations of strongly attractive particles, J. Chem. Phys. 127 (15) (2007) 154101. doi:10.1063/1.2790421.

[15] L. Rovigatti, J. Russo, F. Romano, How to simulate patchy particles, Eur. Phys. J. E 41 (5) (2018) 59. doi:10.1140/epje/i2018-11667-x.

[16] T. Preis, P. Virnau, W. Paul, J. J. Schneider, GPU accelerated Monte Carlo simulation of the 2d and 3d Ising model, J. Comput. Phys. 228 (12) (2009) 4468–4477. doi:10.1016/j.jcp.2009.03.018.

[17] J. A. Anderson, E. Jankowski, T. L. Grubb, M. Engel, S. C. Glotzer, Massively parallel Monte Carlo for many-particle simulations on GPUs, J. Comput. Phys. 254 (2013) 27–38. doi:10.1016/j.jcp.2013.07.023.

[18] B. Chen, J. I. Siepmann, A novel Monte Carlo algorithm for simulating strongly associating fluids: Applications to water, hydrogen fluoride, and acetic acid, J. Phys. Chem. B 104 (36) (2000) 8725–8734. doi:10.1021/jp001952u.

[19] B. Chen, J. I. Siepmann, K. J. Oh, M. L. Klein, Aggregation-volume-bias Monte Carlo simulations of vapor-liquid nucleation barriers for Lennard-Jonesium, J. Chem. Phys. 115 (23) (2001) 10903–10913. doi:10.1063/1.1417536.

[20] G. Andrade, G. Ramos, D. Madeira, R. Sachetto, R. Ferreira, L. Rocha, G-DBSCAN: A GPU Accelerated Algorithm for Density-based Clustering, Procedia Comput. Sci. 18 (2013) 369–378. doi:10.1016/j.procs.2013.05.200.

[21] Y. Nejahi, M. S. Barhaghi, J. Mick, B. Jackman, K. Rushaidat, Y. Li, L. Schwiebert, J. J. Potoff, GOMC: GPU optimized monte carlo for the simulation of phase equilibria and physical properties of complex fluids, SoftwareX 9 (2019) 20–27. doi:10.1016/j.softx.2018.11.005.

[22] Y. Nejahi, M. S. Barhaghi, G. Schwing, L. Schwiebert, J. J. Potoff, Update 2.70 to "GOMC: GPU optimized monte carlo for the simulation of phase equilibria and physical properties of complex fluids", SoftwareX 13 (2021) 100627. doi:10.1016/j.softx.2020.100627.

[23] J. A. Anderson, M. E. Irrgang, S. C. Glotzer, Scalable metropolis monte carlo for simulation of hard shapes, Computer Physics Communications 204 (2016) 21–30. doi:10.1016/j.cpc.2016.02.024.

[24] E. G. Noya, MC_GPU: GPU-Accelerated Monte Carlo for Patchy Particles, Available at: https://github.com/evanoya/MC_GPU (2021).

[25] E. G. Noya, C. K. Wong, P. Llombart, J. P. K. Doye, How to design an icosahedral quasicrystal through directional bonding, Nature 596 (7872) (2021) 367–371. doi:10.1038/s41586-021-03700-2.

[26] E. G. Noya, J. P. K. Doye, A one-component patchy-particle icosahedral quasicrystal, ACS Nano 19 (14) (2025) 13714–13722. doi:10.1021/acsnano.4c14885.

[27] A. Kowaguchi, S. Mehta, J. P. K. Doye, E. G. Noya, A patchy-particle three-dimensional octagonal quasicrystal, The Journal of Chemical Physics 163 (24) (2025) 244904. doi:10.1063/5.0292922.

[28] A. Vrabecz, G. Tóth, Simulation of binary hard-sphere systems with 1:5 and 1:10 size ratios, Mol. Phys. 104 (12) (2006) 1843–1853. doi:10.1080/00268970600602313.

[29] S. Duane, A. D. Kennedy, B. J. Pendleton, D. Roweth, Hybrid Monte Carlo, Phys. Lett. B 195 (2) (1987) 216–222. doi:10.1016/0370-2693(87)91197-X.

[30] B. Mehlig, D. W. Heermann, B. M. Forrest, Hybrid Monte Carlo method for condensed-matter systems, Phys. Rev. B 45 (1) (1992) 679–685. doi:10.1103/PhysRevB.45.679.

[31] E. Lomba, E. G. Noya, gpMC: A General Purpose Monte Carlo Program, Available at: https://github.com/elomba/MCcol.git (2026).

[32] M. N. Bannerman, R. Sargant, L. Lue, DynamO: A free $O(n)$ general event-driven molecular dynamics simulator, Journal of Computational Chemistry 32 (15) (2011) 3329–3338. doi:10.1002/jcc.21915.